

Modul Praktikum 5: Stateful Architecture & Data Persistence

Topik: Integrasi Managed Database (DBaaS) dan Object Storage melalui Pengembangan Aplikasi Mandiri (Bring Your Own Tech Stack)

1. Pendahuluan

Skenario Studi Kasus: "Portal Pendaftaran Pegawai"

Perusahaan Anda sedang membangun "Portal Pendaftaran Pegawai Baru". Pelamar harus mengisi form **Data Diri (Nama, Email)** dan mengunggah dokumen **Berkas Pelamar (Foto KTP/CV)**.

Karena aplikasi ini nantinya akan dijalankan di lingkungan *Cloud* yang dinamis (seperti *Container* dengan *Auto Scaling*), Anda **dilarang keras** menyimpan data teks maupun foto KTP pelamar di dalam *local storage* atau *hard disk server* aplikasi. Jika *server* mati atau digandakan, data tersebut akan hilang atau tidak sinkron.

Solusi Arsitektur yang Wajib Anda Terapkan:

1. **Database as a Service (DBaaS):** Digunakan murni untuk menyimpan data terstruktur (Nama, Email, dan Link URL Foto).
2. **Object Storage:** Digunakan khusus untuk menyimpan *file* fisik berukuran besar (Foto KTP/CV).

Tujuan Pembelajaran

1. Merancang aplikasi *stateless* yang bergantung pada layanan penyimpanan awan eksternal.
2. Membaca dokumentasi SDK/API *Cloud* untuk mengintegrasikan Object Storage dengan bahasa pemrograman pilihan (Node.js/PHP/Python/Java, dll).
3. Menerapkan praktik keamanan dasar dengan memisahkan *credential* (kunci rahasia) dari kode sumber menggunakan *Environment Variables* (.env).

2. Spesifikasi Kebutuhan Sistem (System Requirements)

Sebagai *Cloud Engineer* sekaligus *Developer*, Anda dibebaskan memilih bahasa pemrograman dan *framework* apa pun. Namun, aplikasi Anda **wajib** memenuhi kriteria alur kerja berikut:

A. Kebutuhan Antarmuka (Frontend):

- Memiliki sebuah halaman web berisi form pendaftaran.
- Form wajib memuat *input*: Nama Lengkap, Alamat Email, dan *Upload File* (Gambar).
- Tombol *Submit* untuk mengirim data.

B. Kebutuhan Logika Sistem (Backend): Saat tombol *Submit* ditekan, aplikasi *backend* Anda harus melakukan dua hal secara berurutan:

1. **Mengunggah File ke Object Storage:** Aplikasi menembak API *Cloud Object Storage* untuk mengunggah *file* gambar. Setelah berhasil, aplikasi harus mendapatkan "URL Publik" dari gambar tersebut.

2. **Menyimpan Teks ke Database:** Aplikasi melakukan koneksi ke DBaaS (MySQL/PostgreSQL) dan melakukan perintah INSERT untuk menyimpan: Nama, Email, dan **URL Publik Gambar** (dari proses nomor 1) ke dalam tabel *database*.

C. Kebutuhan Keamanan (Security):

- Kunci rahasia *Database* (Host, User, Password) dan kunci Object Storage (Access Key/API Key) **TIDAK BOLEH** ditulis langsung (*hardcoded*) di dalam file kode sumber. Wajib dipanggil menggunakan sistem *Environment Variables*.

3. Langkah Kerja (Lab Walkthrough)

Langkah 1: Provisioning Managed Database (Cloud)

1. Buat instans **Managed Database** (MySQL atau PostgreSQL) di konsol *Cloud Provider* pilihan Anda.
2. Pastikan Anda mengatur *Firewall/Network Rules* agar *database* dapat menerima koneksi dari IP Publik laptop/aplikasi Anda.
3. Gunakan *Database Client* (seperti DBeaver, phpMyAdmin, atau TablePlus) untuk me-remote database tersebut dan buatlah tabel sederhana (misal: tabel pelamar dengan kolom id, nama, email, ktp_url).
4. Catat *Endpoint/Host, Port, Username, dan Password*.

Langkah 2: Provisioning Object Storage (Cloud)

1. Buat *Bucket* Object Storage baru (S3 / GCS / Azure Blob) dengan nama yang unik.
2. Atur *Bucket Policy* atau *Access Control List (ACL)* agar *file* yang diunggah ke dalamnya bisa dibaca oleh publik (agar URL fotonya nanti bisa dibuka di *browser*).
3. Buat/Generate **Access Key** atau **Service Account** yang memiliki izin (*permission*) untuk menulis/mengunggah data ke *Bucket* tersebut. Catat kredensial ini.

Langkah 3: Pengembangan Kode (Lokal)

1. Buat proyek aplikasi di laptop Anda menggunakan *tech stack* favorit Anda.
2. Instal *library/SDK Cloud* yang sesuai dengan bahasa pemrograman Anda (misalnya: boto3 untuk Python, aws-sdk untuk Node.js, atau google-cloud-storage).
3. Tulis kode program sesuai dengan **Spesifikasi Kebutuhan Sistem** di Bagian 2.
4. Lakukan pengujian di laptop Anda (*localhost*).

Langkah 4: Pengujian End-to-End

1. Buka aplikasi web Anda, isi data diri, dan unggah sebuah foto *dummy*. Klik *Submit*.
2. Jika tidak ada *error*, buka *Database Client* Anda dan jalankan kueri SELECT * FROM pelamar. Pastikan data teks dan URL berhasil masuk.
3. Salin *URL gambar* dari *database* tersebut, lalu *paste* di *browser* baru. Pastikan gambar KTP yang Anda unggah berhasil ditampilkan (membuktikan integrasi Object Storage sukses).

4. Luaran Praktikum (Deliverables) & Analisis

Kumpulkan laporan berformat PDF yang memuat:

1. Bukti Fungsionalitas (Tangkapan Layar):

- *Screenshot* antarmuka form aplikasi Anda saat sedang diisi.
- *Screenshot* tabel di dalam **DBaaS Cloud** yang memperlihatkan *record* data pelamar (termasuk *URL Object Storage* yang tersimpan di kolom tabel).
- *Screenshot* halaman konsol **Object Storage (Bucket)** di *Cloud* yang memperlihatkan *file* gambar fisik telah berhasil bersarang di sana.

2. Bukti Kode Sumber:

- Lampirkan **Tautan (Link) Repository GitHub** publik yang berisi kode aplikasi Anda. (Dosen/Asisten akan mengecek apakah Anda benar-benar menggunakan SDK *Cloud* dan *.env*).

3. Analisis Arsitektur: Dalam arsitektur tradisional, *file* gambar (KTP) sering kali diunggah dan diubah menjadi tipe data BLOB (Binary Large Object) untuk disimpan langsung di dalam *database* relasional (MySQL) agar menyatu dengan data teks. Namun, di arsitektur *Cloud Native* yang Anda praktikkan hari ini, kita dengan sengaja memisahkan *file* gambar ke *Object Storage*, dan *Database* hanya menyimpan *Link URL*-nya saja. **Jelaskan minimal 2 (dua) kerugian fatal jika kita memaksakan diri menyimpan *file* gambar/video ke dalam *Database* (sebagai BLOB) pada aplikasi yang memiliki ribuan pengguna aktif! (Tinjau dari segi biaya komputasi *Cloud* dan performa/kecepatan).**